

Team Number: 22

Team Members (SRNs): [PES1UG24AM008], [PES1UG24AM013]

Section: AIML A

Project Synopsis: Vehicle Service & Maintenance Platform

1. Introduction

Vehicle servicing and maintenance involves several interconnected activities such as customer management, vehicle records, service requests, mechanic assignment, spare-part management, billing, payments, and service history. In many small and medium-sized service centres, these activities are handled using manual records, spreadsheets, or disconnected systems. This can lead to difficulties in tracking vehicle history, managing mechanic availability, maintaining accurate spare-part inventory, and handling multiple service requests simultaneously.

The proposed **Vehicle Service & Maintenance Platform** is a database-driven web application designed to organize and manage these activities through a centralized system. The system will allow customers to register their vehicles and request services, while service-center staff can manage mechanics, service requests, spare parts, service records, invoices, and payments.

The project will focus particularly on database-related problems such as **service scheduling, concurrent mechanic allocation, inventory consistency, transaction management, and service-history analysis**, making it more than a basic CRUD application.

2. Problem Statement

A vehicle service centre must coordinate multiple customers, vehicles, mechanics, service requests, spare parts, and payments. When several service requests arrive at the same time, assigning a limited number of mechanics becomes a scheduling and concurrency problem. Similarly, simultaneous use of spare parts can result in incorrect inventory values if database operations are not handled properly.

The proposed system aims to solve these problems by providing a centralized database-backed platform that maintains data consistency, supports concurrent operations, manages service workflows, and provides useful analytics from historical service data.

3. Objectives

The main objectives of the project are:

- To design and implement a relational database for managing customers, vehicles, mechanics, service requests, service records, spare parts, inventory, invoices, and payments.

- To provide a web-based interface using Next.js for customers, mechanics, and service-center managers.
- To implement service-request scheduling and mechanic assignment while handling concurrent requests.
- To maintain accurate spare-part inventory using transactions and database constraints.
- To provide vehicle service history and service-cost analytics.
- To implement database features such as joins, subqueries, views, triggers, stored procedures/functions, indexes, CTEs, and window functions.
- To use NoSQL storage for flexible service/diagnostic information where appropriate.
- To implement semantic search over previous service descriptions using vector embeddings and similarity search.
- To maintain data integrity, validation, authentication, authorization, and secure database configuration.

4. Proposed System

The system will contain three main user roles:

Customer

Customers can register and log in, add their vehicles, submit service requests, view service status and service history, view invoices, and provide service reviews.

Mechanic

Mechanics can view their assigned service requests, update service progress, record diagnosis and work performed, and record spare parts used during a service.

Service Manager

Managers can view incoming service requests, assign available mechanics, manage spare-part inventory, monitor mechanic workload, manage service records and invoices, and view service analytics.

The primary workflow will be:

Customer → Vehicle → Service Request → Mechanic Assignment → Service Record → Parts Used → Invoice → Payment

The system will also maintain the complete service history of each vehicle.

5. Database Design and Technologies

The core structured data will be implemented using **PostgreSQL**. The relational database will contain entities such as Customer, Vehicle, Service-Centre, Mechanic, ServiceRequest, Service Record, Spare Part, Inventory, Invoice, Payment, and a junction entity for parts used during a service.

An **ER diagram** will be developed and converted into a relational schema containing primary keys, foreign keys, constraints, and appropriate relationships. The database will also be normalized using functional dependencies and normal forms such as 1NF, 2NF, and 3NF.

MongoDB will be considered for flexible diagnostic information and technician observations whose structure may differ between different types of vehicle services.

A **vector database/vector extension such as pgvector** will be used for semantic search over historical service descriptions. For example, a new description such as "engine makes a rattling sound during acceleration" could retrieve previous service records containing semantically similar descriptions even when the exact words are different.

The project requirements specifically allow SQL, NoSQL, and vector databases and require their use to be justified according to the nature of the application data.

6. Advanced Database Features

The project will demonstrate several advanced DBMS concepts.

A major feature will be **transaction and concurrency management**. For example, if two mechanics are available and three service requests arrive simultaneously, the system must prevent the same mechanic from being assigned to multiple requests. Transactions and row-level locking will be used to maintain consistency.

Transactions will also be used when updating inventory and assigning service resources so that related operations either complete successfully or are rolled back.

Other database features will include:

- Views for vehicle service history.
- Triggers for maintaining inventory consistency.
- Stored procedures/functions for service-related operations.
- Indexes for frequently searched fields.
- Aggregate queries for service and revenue analysis.
- CTEs and window functions for mechanic and service-center analytics.
- Full-text or semantic search for service descriptions.
- NoSQL aggregation where appropriate.

The project requirements specify that at least four advanced database concepts such as transactions, views, procedures/functions, triggers, indexing, full-text search, NoSQL aggregation, or vector similarity search should be demonstrated.

7. Application and Expected Outcome

The frontend will be developed using **Next.js**, with API/route handlers providing communication between the application and databases. Database credentials will be stored using environment variables rather than hard-coded in the application, following the project requirements.

The final system is expected to provide a functional platform for managing the complete vehicle-service lifecycle while demonstrating practical DBMS concepts. The project will show how relational databases can maintain structured transactional data, NoSQL can handle flexible information, and vector search can support semantic retrieval of historical service cases.

The expected outcome is a working web application that goes beyond simple CRUD operations and demonstrates the practical application of **database design, SQL, normalization, query processing, transactions, concurrency control, NoSQL, vector search, and database–application integration** in a real-world vehicle service scenario.